

Documentación Técnica: Sistema FSM

Este documento detalla la arquitectura y el funcionamiento del sistema de Máquina de Estados Finitos (FSM) implementado para controlar el comportamiento y la lógica de combate de las entidades. El sistema está diseñado con una arquitectura modular y escalable, utilizando un enfoque similar a un *Blackboard* para la gestión de variables globales y estados fuertemente desacoplados.

1. Arquitectura Central del Sistema

El núcleo del comportamiento de la IA se divide en cuatro componentes principales que interactúan entre sí para asegurar un flujo de ejecución limpio y basado en las mejores prácticas de programación orientada a objetos.

VisceralStateMachine (El Controlador)

Actúa como el "cerebro" central de la entidad. Este componente se encarga de:

- **Almacenamiento de Estados:** Mantiene un arreglo con todos los **BaseState** disponibles para la entidad.
- **Diccionario de Condiciones (Blackboard):** Gestiona un diccionario global de tipo `<string, bool>`. Este diccionario dicta las reglas bajo las cuales los estados pueden transicionar (ej. `Melee = true`).
- **Ciclo de Ejecución:** Llama a la función `OnTick()` del estado actual en cada frame mediante el método `Update()` de Unity.
- **Gestión de Transiciones:** Si el estado actual indica que sus condiciones de salida se cumplen, ejecuta `SwitchToNewState()`, llamando a `OnExit()` del estado anterior y a `OnEnter()` del nuevo. Además, resetea variables globales si el flag `IsResetAfterSwitch` está activo.

BaseState (Plantilla de Estado)

Es la clase base monobehaviour de la cual heredan todos los estados específicos. Define el ciclo de vida estándar de un estado:

- `OnInitialize()`: Se ejecuta una vez al inicio para cachear referencias (como el controlador de animaciones o componentes de movimiento).
- `OnEnter()` / `OnExit()`: Lógica de entrada y salida al transicionar hacia o desde este estado.
- `OnTick()`: La lógica que se ejecuta frame a frame (como el movimiento o el chequeo de animaciones).

- `EvaluateTransitions()`: Itera sobre el arreglo de `Transition` del estado para verificar si debe cambiar a otro.

2. Lógica de Transiciones y Condiciones

El sistema utiliza una evaluación dinámica para decidir cuándo cambiar de comportamiento.

Estructura de la Transición: Una **Transition** conecta un estado de origen (`FromGO`) con un estado de destino (`ToGO`). Para que ocurra, todas las **Condition** asignadas deben coincidir con su respectivo valor en el diccionario global de la **VisceralStateMachine**.

Guía Visual en el Inspector

A continuación se observa la configuración para salir del estado de Movimiento para entrar en Combate:

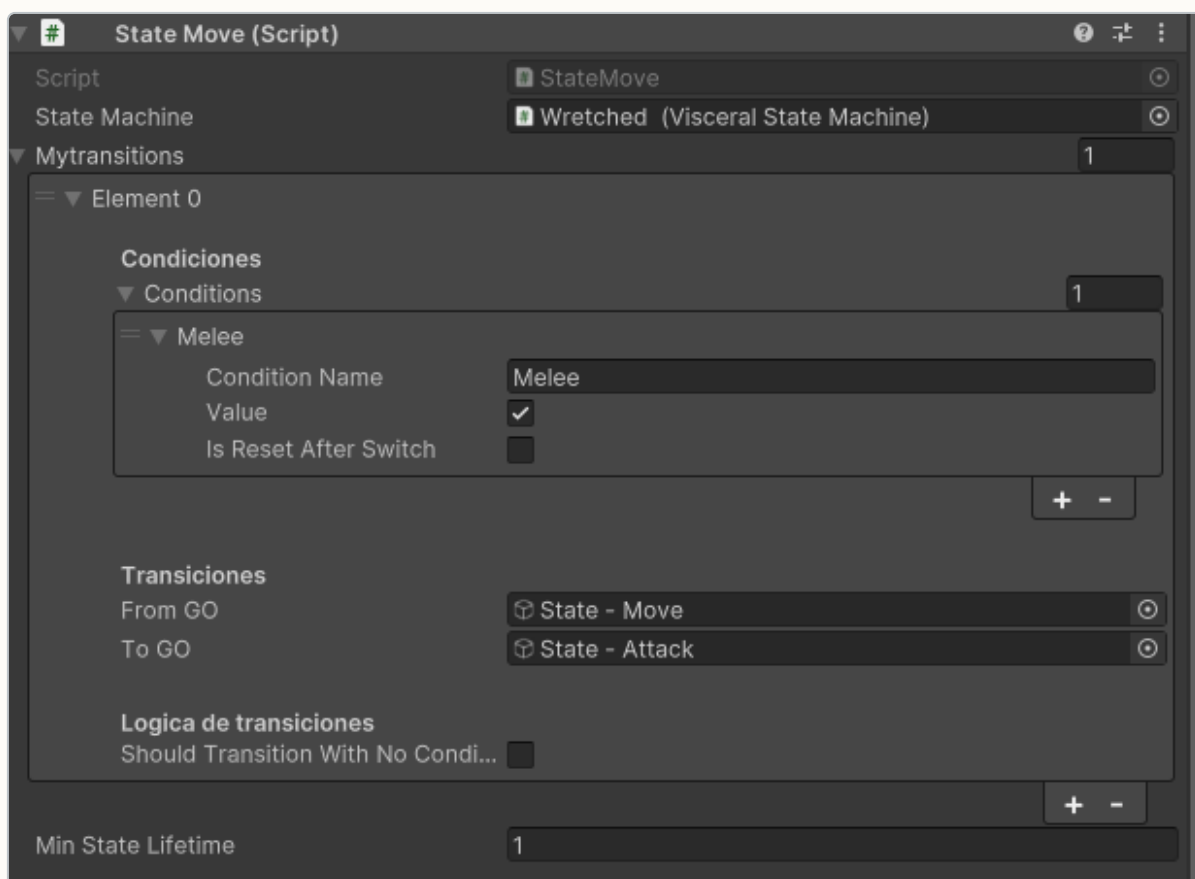


Figura 1: Transición basada en condiciones. El estado "State_Move" evalúa continuamente el parámetro global "Melee". Cuando el valor es True, aprueba la transición hacia "State_Attack".

Por otro lado, existen transiciones **garantizadas o incondicionales**. Si el flag `ShouldTransitionWithNoConditions` está activado, la transición ignorará la lista de condiciones y

forzará el cambio, ideal para encadenar estados cuando una animación o ciclo ha terminado obligatoriamente.

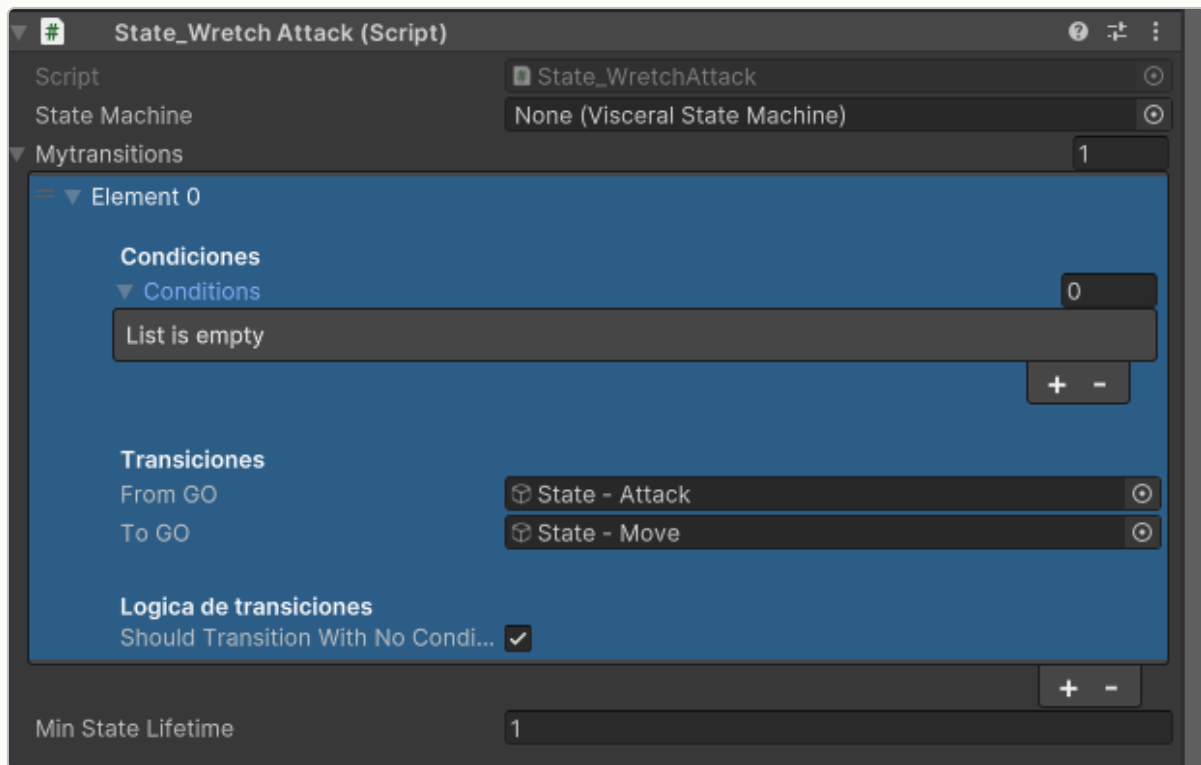


Figura 2: Transición incondicional. En el estado de ataque, se observa que "Should Transition With No Conditions" está marcado. Una vez que la lógica interna de `State\_WretchAttack` decide terminar (ej. terminó su ataque), forzará el regreso a "State_Move" sin requerir la evaluación de booleanos globales.

3. Caso de Estudio: `State_WretchAttack`

Este script representa la lógica para un enemigo tipo cargador (charger) en Visceral Limbo. Aprovecha el sistema FSM implementando mecánicas robustas de predicción de movimiento y evasión de obstáculos.

Flujo de Ataque

- **Inicio (OnEnter):** Al entrar, se calcula si el ataque será normal o predictivo basado en `_ChanceForPredictiveAttack`. Activa el parámetro de animación de ataque e informa al `IMovementStrategy` que evite obstáculos.
- **Ataque Predictivo (PredictiveAttack):** Obtiene la velocidad actual del jugador (usando la interfaz `KinematicCharacterMotor`) y la multiplica por un `_PredictiveOffset`. Esto genera un vector de dirección adelantado al objetivo, simulando una IA de intercepción inteligente que busca cortar el paso del jugador.
- **Ejecución (OnTick):**
 - **Windup:** Mientras se reproduce la animación previa al ataque, la velocidad de movimiento se anula (el enemigo se "carga").

- **Embestida (Start/Idle Attack):** Lanza un Raycast hacia adelante usando `IsHittingWall()`. Si detecta una pared, detiene el movimiento de golpe y desactiva la evasión de obstáculos para evitar comportamientos erráticos al chocar. Si la ruta está libre, avanza con fuerza (`_AttackMovementStrenght`).
- **Recuperación (EndAttack):** Reduce progresivamente la velocidad usando un `Mathf.Lerp` en base al tiempo normalizado de la animación de finalización, logrando un frenado suave. Cuando llega al final, levanta la bandera `_FinishedAttack`.
- **Finalización (EvaluateTransitions / OnExit):** Cuando `_FinishedAttack = true`, el FSM permite la evaluación. Dado que está configurado como transición incondicional (Figura 2), pasará automáticamente al estado definido (Move). El `OnExit` limpia las velocidades y vuelve a resetear el parámetro global `Melee` a `true` para dejarlo listo para el próximo ciclo.